

Informe - Etapa 1

Autor: Nicolas Cid
Materia: Compiladores e Intérpretes

Tokens y sus Expresiones Regulares

Definiciones auxiliares

Definiciones usadas para simplificar la escritura y lectura de las expresiones regulares.

- `Letra = [a..z] | [A..Z]`
 - `LetraMayus = [A..Z]`
 - `LetraMinus = [a..z]`
 - `Digito = [0..9]`
 - `Caracter = Letra | Digito | _`
 - `CaracterVisible = ASCII [32 - 126]`
 - `CaracterString = CaracterVisible - { " , \ }`
 - `CaracterChar = CaracterVisible - { ' , \ }`
-

Palabras reservadas

Token	Expresión
<code>prClass</code>	<code>class</code>
<code>prBoolean</code>	<code>boolean</code>
<code>prIf</code>	<code>if</code>
<code>prThis</code>	<code>this</code>
<code>prExtends</code>	<code>extends</code>
<code>prChar</code>	<code>char</code>
<code>prElse</code>	<code>else</code>
<code>prNew</code>	<code>new</code>
<code>prInterface</code>	<code>interface</code>
<code>prInt</code>	<code>int</code>
<code>prWhile</code>	<code>while</code>
<code>prNull</code>	<code>null</code>
<code>prImplements</code>	<code>implements</code>

Token	Expresión
prVoid	void
prReturn	return
prTrue	true
prStatic	static
prPublic	public
prVar	var
prFalse	false

Identificadores

Token	Expresión
idClase	LetraMayus(Caracter)*
idGen	LetraMayus
idMetVal	LetraMinus(Caracter)*

Literales

Token	Expresión
intLiteral	Digito{1,9}
charLiteral	'CaracterChar' '\CaracterVisible'
stringLiteral	"CaracterString*" "((CaracterString)* \CaracterVisible)*"

Símbolos de puntuación

Token	Símbolo
puParentesisAbre	(
puParentesisCierra	)
puLlaveAbre	{
puLlaveCierra	}
puCorcheteAbre	[
puCorcheteCierra	]
puPuntoYComa	;
puComa	,
puPunto	.

Token	Símbolo
puDosPuntos	:

Símbolos operadores

Token	Símbolo
opMayor	>
opMenor	<
opNegacion	!
opAsignacion	=
opIgualdad	==
opMayorIgual	>=
opMenorIgual	<=
opDistinto	!=
opAnd	&&
opOr	
opModulo	%
opSuma	+
opResta	-
opMultiplicacion	*
opDivision	/
opIncremento	++
opDecremento	--

End of file

EOF = (char) 26

EOF no está expresado como una expresión regular. Es literalmente el carácter número 26 de la tabla ASCII.

Aclaraciones

Escape dentro de literales string y literales char

Se tomó la convención de permitir escapar cualquier caracter visible (ASCII [32-126]).

Esto permite que sean válidos strings como:

- `"\\"`
- `"\@"`
- `"\a"`

En Java algunos, como `"\\"`, son válidos; otros, como `"\@"`, no.

Casos como `"\\\""` caen dentro del error de **string mal cerrado**, ya que se está escapando a la doble comilla.

Otro caso similar, pero distinto, es `"\\\ "`. Si bien hay una barra invertida que pareciera estar "suelta", en realidad está escapando a un espacio en blanco (`" "`), por lo que resulta un string válido bajo la convención adoptada.

Créditos de casos de prueba compartidos

- Tomás Bertotto
- Santiago Salamanca

Dentro del directorio `resources/` se pueden encontrar los casos de prueba provistos por la cátedra en sus directorios originales (`conErrores/`, `sinErrores/`) junto con los creados por el autor (con la asistencia de Claude).

Los casos de prueba cedidos por compañeros se encuentran en directorios con su nombre y apellido, y los Testers están nombrados con la misma convención que los originales provistos por la cátedra, con la adición de las iniciales del autor de los casos de prueba correspondientes.

Instrucciones de Compilación y Uso

Requisitos previos

- JDK 21 instalado
- No requiere tener Gradle instalado previamente (el proyecto incluye el Gradle Wrapper)

Compilación

Desde la raíz del proyecto, ejecutar:

Linux / macOS: `./gradlew jar`

Windows: `gradlew.bat jar`

Esto genera el ejecutable en: `build/libs/Compilador.jar`

Uso

Una vez compilado, el compilador se invoca desde la línea de comandos pasando como parámetro el archivo fuente de MiniJava. El comando es el mismo en Linux, macOS y Windows, ya que se ejecuta a través de `java`:

```
java -jar build/libs/Compilador.jar programa1.java
```

Donde `programa1.java` es el archivo fuente de MiniJava a compilar (se acepta cualquier extensión).

Logros

- Manejador de archivos eficiente